1.- D.El JDK (Java Development Kit) es el kit completo para desarrolladores. Incluye el compilador (javac), herramientas de depuración (jdb), utilidades de empaquetado (jar) y contiene al JRE (Java Runtime Environment) para poder ejecutar las aplicaciones.

2.- E.  jar: Es la herramienta de linea de comandos incluida en el JDK que se utiliza específicamente para crear, listar, extraer y manipular archivos comprimidos en formato .jar (Java Archive).

3.-  D. jconsole: Es una herramienta gráfica de monitoreo y gestión incluida en el JDK que cumple con la especificación JMX (Java Management Extensions). Permite supervisar el uso de memoria (heap), el consumo de CPU, el comportamiento del recolector de basura (Garbage Collector), los hilos en ejecución (threads) y las clases cargadas en tiempo real.

4.- D. Porque boolean es un tipo primitivo que solo acepta true o false y la opción d es la que con el resultado de la evaluación obtenemos un true o false.

5.- C,D, E Y F. De nuevo boolean solo acepta true o false y las opciones seleccionadas a través de una evaluación obtenemos el resultado esperado.

6.- D. Porque el casteo entre tipos numéricos primitivos se permite (casteo implícito).
7.- D. Porque se crea primero la variable ‘a’ pero después se incrementa y en ambos print’s se imprime ‘a’  con la última operación.
8.- Porque se crea primero la variable ‘b’ y se inicializa en 4 pero después se decrementa y en ambos print’s se imprime ‘b’ con el último movimiento que se realizó con la variable’ .
9.- Porque las tablas de verdad de los operadores OR y AND  nos indican que OR solo una vez da ‘false’ y AND solo una vez da ‘false’, aquí se presentan los casos donde AND vale ‘false’ que se le aplica OR que da ‘true’.
10.- true and false = false, false OR true = true, NOT true = false.
11.- En el primer ‘print’ se manda a imprimir en pantalla la concatenación de un número de dos dígitos mientras en el segundo ‘print’ se concatena la suma de dos números.
12.- En el primer ‘print’ se hace una concatenación (+): “result : 235”; mientras que en segundo ‘print’ se traduce como “Result: 2 + (3*5)”-->”Result: 2 + 15”-->”Result: 215”.
13.-  Se hace la conversión binaria.
14.a. boolean flag = Boolean.FALSE; (Válida por unboxing: el objeto envoltorio Boolean.FALSE se convierte automáticamente al valor primitivo false).

b. boolean flag = true; (Válida: asignación directa del literal booleano).

d. boolean flag = false; (Válida: asignación directa del literal booleano).

15.- En Java, no se pueden sumar variables booleanas. El operador + está reservado exclusivamente para operaciones matemáticas con números y para la concatenación de cadenas de texto (String).
16.- Int a, b, c = 0; (Line 15): Falla porque Java distingue entre mayúsculas y minúsculas (case-sensitive). El tipo de dato primitivo correcto es int (todo en minúsculas), no Int.

int g, int h, int i = 0; (Line 17): Falla por un error de sintaxis. Cuando declaras múltiples variables en una sola línea separadas por comas, solo debes escribir el tipo de dato una vez al principio (debería ser int g, h, i = 0;).

Int k, l, m, = 0; (Line 19): Falla por dos razones: usa la "l" mayúscula en Int, y tiene una coma extra inválida justo antes del signo de igual (m, =).

17.- En Java, un byte es un tipo de dato numérico con signo de 8 bits (puede contener valores negativos de -128 a 127). Por otro lado, un char es un tipo de dato numérico sin signo de 16 bits (solo contiene valores positivos de 0 a 65535, que representan caracteres Unicode).

Debido a que un byte podría contener un valor negativo que un char no es capaz de representar de forma natural, el compilador de Java no permite hacer esta asignación de forma automática (implícita) para protegerte de la pérdida de datos o resultados inesperados.

18.- public static final void main(String[] args): Este es el método que la JVM ejecuta. Cumple con todos los requisitos (es público, estático, devuelve void y recibe un arreglo de String). El modificador final es completamente válido aquí; simplemente significa que este método no podría ser sobrescrito (overridden) si esta clase fuera heredada por otra, pero a la JVM no le molesta al momento de iniciar el programa. Imprime "three".

19.- c. class Fred1 { public static void main(String[] args) { System.out.println(args); } }

Compila: Sí, tiene la firma exacta del método main (String[] args).

Ejecución: Sí, se ejecuta con éxito. Imprime la representación en memoria del arreglo (por ejemplo, algo como [Ljava.lang.String;@...), pero no lanza ninguna excepción.

d. class Fred1 { public static void main(String[] args) { System.out.println(args[1]); } }

Compila: Sí, la firma es correcta.

Ejecución: Sí, se ejecuta con éxito. Al pasar los argumentos hello (índice 0) y walls (índice 1), args[1] accede correctamente al texto "walls" y lo imprime en consola sin errores.

20.- Al multiplicarlos, Java promueve automáticamente el int a long para que coincidan. El resultado de esta sub-expresión es un long.

21.- d. public static void main(String[] args) {}, Es la forma estándar, clásica y universalmente correcta para definir el punto de entrada de una aplicación Java que la JVM (Máquina Virtual de Java) reconocerá y ejecutará sin problemas.

c. public static void main(String... String) {}, Es válida y compila/ejecuta. Desde Java 5, los argumentos variables (varargs, representados con ...) son totalmente compatibles a nivel de bytecode con los arreglos ([]). Por lo tanto, String... es equivalente a String[]. Además, nombrar a la variable String (igual que la clase) es una pésima práctica de código, pero sintácticamente legal en Java (el compilador lo acepta porque las variables y los tipos se distinguen por contexto).

22.- b.Verdadero. Cuando utilizas la opción -jar, el archivo JAR especificado se convierte en la única fuente de clases para la aplicación. La JVM ignora por completo cualquier configuración externa de classpath proveniente de la variable de entorno CLASSPATH o de los argumentos de línea de comandos como -cp o -classpath. Si el JAR necesita dependencias externas, estas deben declararse obligatoriamente dentro del propio manifiesto del JAR mediante la directiva Class-Path.

d. Verdadero. Para que la Máquina Virtual de Java sepa qué clase debe ejecutar al iniciar el archivo comprimido, este archivo de metadatos debe incluir explícitamente el atributo Main-Class apuntando a la clase principal.